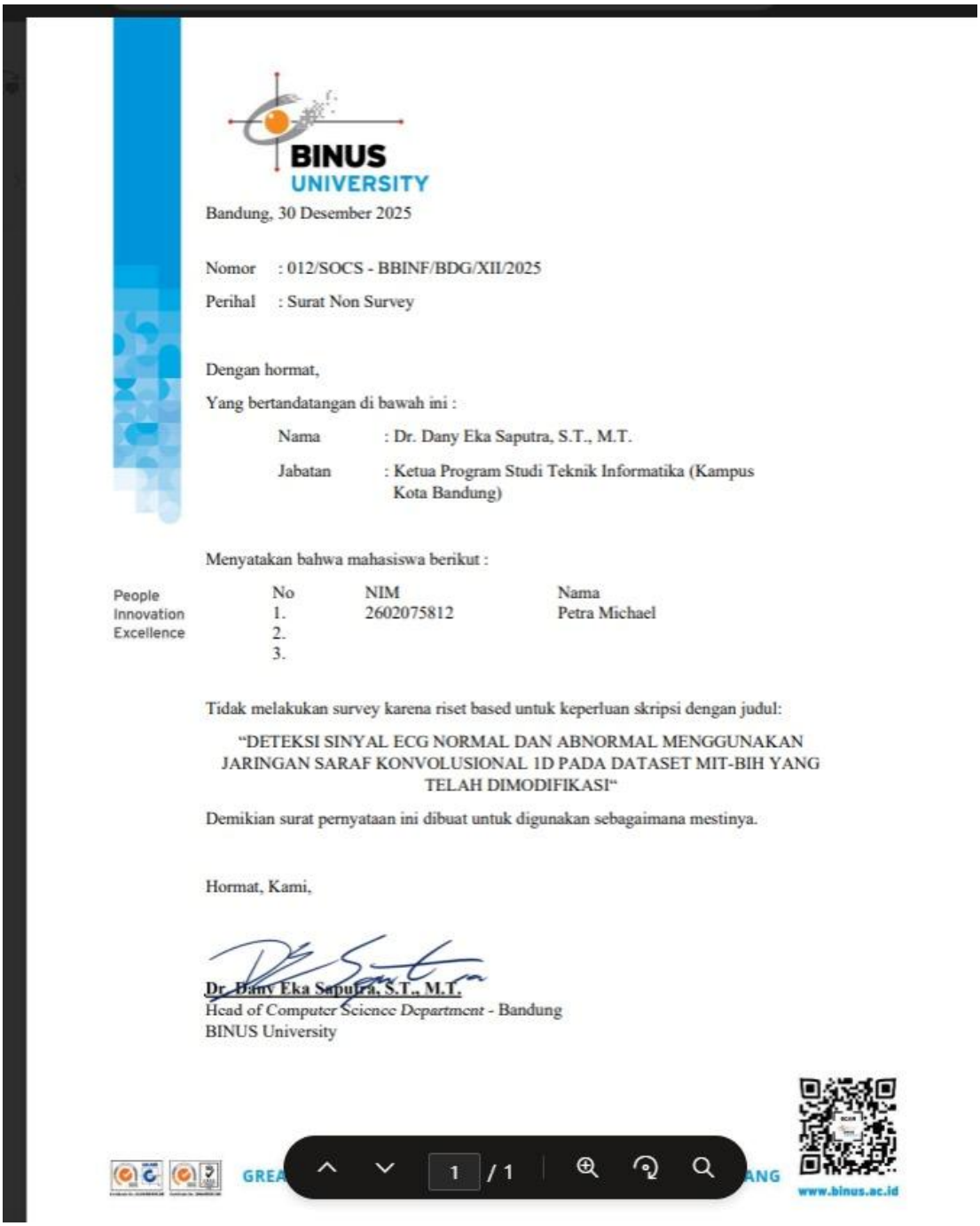
LAMPIRAN

Lampiran 1 Surat Non-Survey



BINUS UNIVERSITY

Bandung, 30 Desember 2025

Nomor : 012/SOCS - BBINF/BDG/XII/2025

Perihal : Surat Non Survey

Dengan hormat,

Yang bertandatangan di bawah ini :

Nama : Dr. Dany Eka Saputra, S.T., M.T.

Jabatan : Ketua Program Studi Teknik Informatika (Kampus Kota Bandung)

Menyatakan bahwa mahasiswa berikut :

People
Innovation
Excellence

No	NIM	Nama
1.	2602075812	Petra Michael
2.		
3.		

Tidak melakukan survey karena riset based untuk keperluan skripsi dengan judul:

"DETEKSI SINYAL ECG NORMAL DAN ABNORMAL MENGGUNAKAN JARINGAN SARAF KONVOLUSIONAL 1D PADA DATASET MIT-BIH YANG TELAH DIMODIFIKASI"

Demikian surat pernyataan ini dibuat untuk digunakan sebagaimana mestinya.

Hormat, Kami,

Dr. Dany Eka Saputra, S.T., M.T.
Head of Computer Science Department - Bandung
BINUS University

GREA 1 / 1 ANG

www.binus.ac.id

Lampiran 2 : Koding untuk import dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix, classification_report,
roc_auc_score, roc_curve,
    accuracy_score, precision_score, recall_score,
f1_score,
    log_loss, mean_absolute_error, mean_squared_error,
r2_score,
    silhouette_score, davies_bouldin_score
)
import os


KAGGLE_PATH = '/kaggle/input/new-
ECG/mitbih_beats_dataset.csv'
LOCAL_PATH = '../dataset/mitbih_beats_dataset.csv'
ALT_PATH = 'mitbih_beats_dataset.csv'
```

Lampiran 3 : Koding untuk proses data agar *No Leakage*

```
sample_cols = [c for c in df.columns if
c.startswith('sample_')]
has_rr = 'rr_interval' in df.columns


if has_rr:
    X_samples = df[sample_cols].values
    X_rr = df['rr_interval'].values.reshape(-1, 1)
    print(f'Using RR interval as additional feature')
else:
    X_samples = df.drop('label', axis=1).values
    X_rr = None
    print(f'No RR interval, using samples only')


y = df['label'].values


print(f'Samples shape: {X_samples.shape}')
print(f'Labels shape: {y.shape}')
```

```
# SPLIT FIRST
if has_rr:
    X_samples_train, X_samples_temp, X_rr_train, X_rr_temp, y_train, y_temp = train_test_split(
        X_samples, X_rr, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
    )
    X_samples_val, X_samples_test, X_rr_val, X_rr_test, y_val, y_test = train_test_split(
        X_samples_temp, X_rr_temp, y_temp, test_size=0.5, random_state=RANDOM_STATE, stratify=y_temp
    )
else:
    X_samples_train, X_samples_temp, y_train, y_temp = train_test_split(
        X_samples, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
    )
    X_samples_val, X_samples_test, y_val, y_test = train_test_split(
        X_samples_temp, y_temp, test_size=0.5, random_state=RANDOM_STATE, stratify=y_temp
    )
    X_rr_train = X_rr_val = X_rr_test = None

print(f'Train: {len(y_train)}, Val: {len(y_val)}, Test: {len(y_test)}')

# NORMALIZE AFTER SPLIT - FIT ONLY ON TRAINING DATA
scaler_samples = StandardScaler()
X_samples_train_norm = scaler_samples.fit_transform(X_samples_train)
X_samples_val_norm = scaler_samples.transform(X_samples_val)
X_samples_test_norm = scaler_samples.transform(X_samples_test)
```

```
if has_rr:
    scaler_rr = StandardScaler()
    X_rr_train_norm =
scaler_rr.fit_transform(X_rr_train)
    X_rr_val_norm = scaler_rr.transform(X_rr_val)
    X_rr_test_norm = scaler_rr.transform(X_rr_test)
else:
    scaler_rr = None


print(f'Normalization (training stats only):')
print(f'  Mean: {X_samples_train_norm.mean():.6f}, Std:
{X_samples_train_norm.std():.6f}')


# Reshape for CNN1D: (batch, channels, length)
X_train_cnn = X_samples_train_norm.reshape(-1, 1, 188)
X_val_cnn = X_samples_val_norm.reshape(-1, 1, 188)
X_test_cnn = X_samples_test_norm.reshape(-1, 1, 188)


print(f'CNN input shape: {X_train_cnn.shape}')
```

Lampiran 4 : Advanced 1D-CNN Koding

```
class ResidualBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size=5,
stride=1, dropout=0.3):
        super().__init__()
        padding = kernel_size // 2
        self.conv1 = nn.Conv1d(in_ch, out_ch,
kernel_size, stride, padding)
        self.bn1 = nn.BatchNorm1d(out_ch)
        self.conv2 = nn.Conv1d(out_ch, out_ch,
kernel_size, 1, padding)
        self.bn2 = nn.BatchNorm1d(out_ch)
        self.dropout = nn.Dropout(dropout)
        self.relu = nn.ReLU()
```

```
        self.shortcut = nn.Sequential()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv1d(in_ch, out_ch, 1, stride),
                nn.BatchNorm1d(out_ch)
            )

    def forward(self, x):
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.dropout(out)
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
        out = self.relu(out)
        return out


class AdvancedCNN1D(nn.Module):
    def __init__(self, use_rr=True, dropout=0.5):
        super().__init__()
        self.use_rr = use_rr

        # CNN backbone
        self.conv1 = nn.Sequential(
            nn.Conv1d(1, 32, kernel_size=7, padding=3),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Dropout(dropout * 0.5)
        )

        self.res1 = ResidualBlock(32, 64,
dropout=dropout * 0.5)
        self.pool1 = nn.MaxPool1d(2)

        self.res2 = ResidualBlock(64, 128,
dropout=dropout * 0.7)
```

```
        self.pool2 = nn.MaxPool1d(2)

        self.res3 = ResidualBlock(128, 256,
dropout=dropout)
        self.global_pool = nn.AdaptiveAvgPool1d(1)

        # Feature dimension after CNN
        cnn_out = 256

        # RR interval branch
        if use_rr:
            self.rr_branch = nn.Sequential(
                nn.Linear(1, 16),
                nn.ReLU(),
                nn.Dropout(dropout * 0.5),
                nn.Linear(16, 32),
                nn.ReLU()
            )
            fc_in = cnn_out + 32
        else:
            fc_in = cnn_out

        # Classifier
        self.classifier = nn.Sequential(
            nn.Linear(fc_in, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(dropout * 0.8),
            nn.Linear(64, 2)
        )

    def forward(self, x_cnn, x_rr=None):
```

```
        # CNN path
        x = self.conv1(x_cnn)
        x = self.pool1(self.res1(x))
        x = self.pool2(self.res2(x))
        x = self.global_pool(self.res3(x))
        x = x.squeeze(-1)

        # Combine with RR if available
        if self.use_rr and x_rr is not None:
            rr_features = self.rr_branch(x_rr)
            x = torch.cat([x, rr_features], dim=1)

        return self.classifier(x)

model = AdvancedCNN1D(use_rr=has_rr,
dropout=0.5).to(device)
print(model)
print(f'Parameters: {sum(p.numel() for p in
model.parameters()):,}')
```

Lampiran 5 : Koding untuk menentukan evaluasi skor akhir

```
model.eval()
all_preds, all_probs, all_labels = [], [], []

with torch.no_grad():
    for batch in test_loader:
        if has_rr:
            x_cnn, x_rr, labels = batch
            x_cnn, x_rr = x_cnn.to(device),
x_rr.to(device)
        else:
            x_cnn, labels = batch
            x_cnn = x_cnn.to(device)
            x_rr = None
```

```
        outputs = model(x_cnn, x_rr)
        probs = torch.softmax(outputs, dim=1)
        _, preds = outputs.max(1)

        all_preds.extend(preds.cpu().numpy())
        all_probs.extend(probs[:, 1].cpu().numpy())
        all_labels.extend(labels.numpy())

all_preds = np.array(all_preds)
all_probs = np.array(all_probs)
all_labels = np.array(all_labels)

# Metrics
accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds, zero_division=0)
recall = recall_score(all_labels, all_preds, zero_division=0)
f1 = f1_score(all_labels, all_preds, zero_division=0)
try:
    auc_roc = roc_auc_score(all_labels, all_probs)
except Exception:
    auc_roc = 0
try:
    logloss = log_loss(all_labels, all_probs)
except Exception:
    logloss = 0

mae = mean_absolute_error(all_labels, all_preds)
mse = mean_squared_error(all_labels, all_preds)
rmse = np.sqrt(mse)

print('=' * 70)
print('COMPREHENSIVE MODEL EVALUATION - Advanced CNN1D (v6-pytorch)')
print('=' * 70)
```

```
print('\nCATEGORY 1: BASIC CLASSIFICATION METRICS')
print('=' * 70)
print(f'1.            ACCURACY:           {accuracy:.4f}
({accuracy*100:.2f}%)')
print(f'2. PRECISION: {precision:.4f}')
print(f'3. RECALL: {recall:.4f}')
print(f'4. F1 SCORE: {f1:.4f}')
print(f'5. LOG LOSS: {logloss:.4f}')
print('\nCATEGORY 2: ROC CURVE AND AUC METRICS')
print('=' * 70)
print(f'6. AUC-ROC: {auc_roc:.4f}')
print('\nCATEGORY 3: ERROR METRICS')
print('=' * 70)
print(f'8a. MAE: {mae:.6f}')
print(f'8b. MSE: {mse:.6f}')
print(f'8c. RMSE: {rmse:.6f}')

print('\n' + '=' * 70)
print('CLASSIFICATION REPORT')
print('=' * 70)
print(classification_report(all_labels,       all_preds,
target_names=['Normal (0)', 'Abnormal (1)']))

print('\nSUMMARY')
print('=' * 70)
print(f'Accuracy:    {accuracy:.4f}')
print(f'Precision:   {precision:.4f}')
print(f'Recall:      {recall:.4f}')
print(f'F1 Score:    {f1:.4f}')
print(f'AUC-ROC:     {auc_roc:.4f}')
```